Supplementary Information

Target-specific sparse correction certificates recover stochastic language trajectories across numerical precision

Lei Ke

Supplementary Note 1: Official SparSamp compatibility reproduction

Artifact provenance.

The reproduction used the revised SparSamp artifact from Zenodo record 15025436. The downloaded archive `Artifact new.zip` had verified MD5 `1CF080219F9F24D0C608151CEE26E99E`. The runner imported the artifact's `encode_spar` and `decode_spar` functions without changing their algorithm implementation. The checkpoint records SHA-256 hashes for `sparsamp.py`, `utils.py`, `get_statistics.py` and `message.txt`, together with a fingerprint of the local GPT-2 directory.

The Basic Test generated 105 tokens, embedded 576 bits (5.486 bits/token) and recovered the complete message. The full matrix used the ten block lengths from published Table 2 and the three top-p values from Tables 3-4. The shared block-64, top-p=1.0 condition was executed once, giving 12 unique configurations and 1,200 configured context trials.

Context and dependency boundary.

The paper reports 100 IMDB contexts. The artifact `get_statistics.py` comment also says 100, but line 49 samples 10 contexts when the context file contains more than 100. The reproduction followed the paper and selected 100 contexts once with public seed 42. Each trial used artifact seed 777, 100-200 generated tokens and the artifact message starting at offset 19.

This is a compatibility reproduction. It used Python 3.11.15, PyTorch 2.7.1+cu126, CUDA 12.6, Transformers 4.41.2 and an NVIDIA GeForce RTX 3060 Laptop GPU. Transformers 4.57.6 produced `AttributeError: 'list' object has no attribute 'get_seq_length'` at the artifact's legacy cache interface. A separate Torch 2.2.2 CUDA environment was attempted, but the upstream wheel transfer repeatedly terminated with TLS or connection-reset errors. No strict Torch 2.2.2 result is claimed.

Outcome definitions.

Token Ambiguity was detected by decoding each generated token sequence to public text and tokenizing it again. Completed trials with ambiguity stayed in the trial ledger but were excluded from the paper's no-ambiguity decoding and capacity denominators. A trial that reached the 200-token ceiling before completing an integer message block was recorded as budget exhausted, matching the artifact's statement that this is not an embedding error.

Embedding rate was aggregated as total encoded bits divided by total generated tokens. Entropy utilization was total encoded bits divided by total model entropy. The core reproduction gate required exact decoding for every completed no-ambiguity trial and no more than 5% relative error for each published Table 2 utilization value and the Table 4 SparSamp embedding-rate and utilization values. Confidence intervals used 10,000

context bootstrap resamples with public seed 20260721. Timing was descriptive and had no acceptance threshold because the GPU differed from the paper.

Results.

The matrix completed 1,193 of 1,200 configured trials. Three block-512 trials and four block-1023 trials exhausted the 200-token budget. Among completed trials, 347 had Token Ambiguity and 846 were eligible for the no-ambiguity analysis. All 846 eligible trials decoded exactly. All 16 capacity comparisons passed the 5% relative-error gate; the maximum relative error was 4.12% for block length 64, top-p=1.0 embedding rate.

The Table 2 utilization curve closely followed the published trend (Supplementary Fig. 1). Observed utilization rose from 0.272 at block length 2 to 0.872 at 32 and 0.986 at 128. The block-1023 eligible aggregate was 1.000 with a context-bootstrap interval of 0.987-1.013, compared with the published value 0.995. A finite sample can produce an aggregate embedded-bits to entropy ratio slightly above one; this observation is not interpreted as exceeding expected information-theoretic capacity.

At block length 64, the observed embedding rates were 3.672, 5.032 and 5.734 bits/token for top-p=0.8, 0.95 and 1.0, compared with published values 3.60, 5.16 and 5.98. Corresponding utilization values were 0.963, 0.953 and 0.943, compared with 0.953, 0.949 and 0.974. These differences all remained within the prespecified 5% relative-error tolerance.

The official reproduction therefore receives `PASS_WITH_LIMITATIONS`: the decoding and capacity gates passed, while seven large-block trials exceeded the fixed generation budget and the software environment was compatible rather than strict. Absolute speed is not used to support artifact fidelity.

Supplementary Table 1

The complete machine-readable table is provided in `source_data/supplementary_table_s1_official.csv`. It reports configured completion, budget exhaustion, Token Ambiguity, eligible decoding, embedding rate, utilization with bootstrap interval, sampling time, time ratio, generation speed and embedding/decoding speed for every configuration.

Reproducibility boundary

The raw 1,200-trial checkpoint is excluded from Git with other model outputs. Every trial was written atomically, and resuming required an exact configuration match. The tracked analysis file stores the raw-checkpoint SHA-256, software environment, bootstrap settings, configuration summaries and acceptance details. Reproduction of the exact timing numbers requires the recorded GPU and software stack; reproduction of the capacity comparison requires the unchanged artifact code, the recorded contexts and the local GPT-2 checkpoint identified by the stored fingerprint.

Supplementary Note 2: Mechanism and trust-boundary comparison

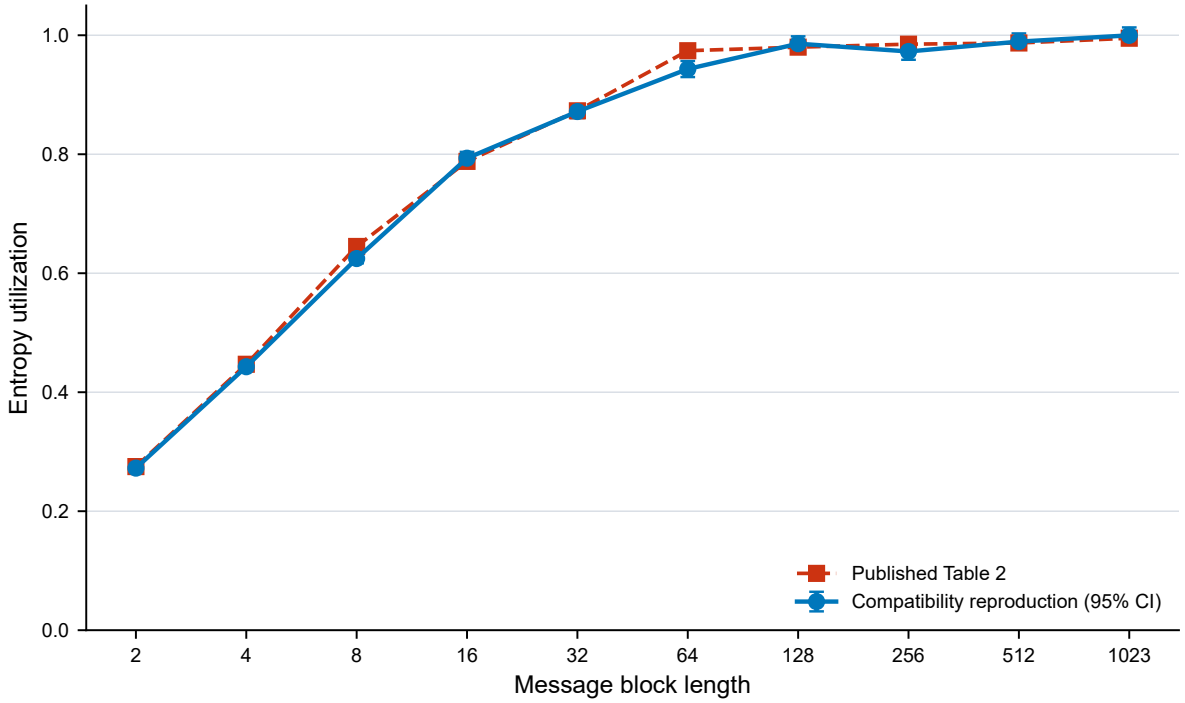
The main study concerns target-specific token-trajectory replay, not secret-message transmission. Table S2 separates adjacent mechanisms by objective and state assumptions so that the SparSamp compatibility result is not treated as inherited evidence for SPRC novelty.

Table S2: Mechanism and trust-boundary comparison.

Mechanism	Primary objective	Distribution relationship	Recipient state	Exact selected-path replay	Tokenization layer
Public seed only	Re-run stochastic inference	Uses the target implementation's local distribution	Model, prompt, configuration and seed	No; 5/20 on the frozen seed-0 subset	Shared tokenizer only
Full token trace	Store the selected path	Does not regenerate a distribution	Complete token sequence plus tokenizer	Yes, target-independent	Shared tokenizer only
SPRC	Repair a known target's seeded path	Quantized top-k integer contract; truncation and quantization disclosed	Reference bundle, target identity and sparse corrections	Yes, conditional on the constructed target	Shared tokenizer only
Unquantized top-two delta	Isolate the probability-contract contribution	Same HMAC fraction and top-two support without bins or integer mass	Reference bundle, target identity and sparse corrections	Yes, conditional on the constructed target	Shared tokenizer only
Unquantized top-16-cap delta	Probe wider positive support	Up to 16 positive-probability BF16 candidates; one support shortfall observed	Reference bundle, target identity and sparse corrections	Yes, conditional on the constructed target	Shared tokenizer only
Fixed block repair	Repair every block containing a disagreement	Same contract as SPRC	Reference bundle, target identity and dirty reference blocks	Yes, conditional on the constructed target	Shared tokenizer only
SparSamp [3]	Provably secure message embedding under its threat model	Probability-partition sampling claim	Model/context plus message-decoding state	Not its primary objective	Token Ambiguity is a separate condition
Range coding [4]	Efficient secure linguistic coding	Range partition over next-token probabilities	Synchronized coder state	Not its primary objective	Shared tokenization assumed
List decoding [7]	Improve PSS recovery/capacity trade-offs	Candidate-list coding	Decoder list/search state	Not evaluated here	Shared tokenization assumed
Dyadic approximation [8]	Approximate LLM output laws with dyadic codes	Explicit rate-distribution approximation objective	Dyadic code specification	Not evaluated here	Shared tokenization assumed
ReTokSync [9]	Recover synchronization after text re-tokenization	Orthogonal to probability replay	Public text and synchronization layer	Not evaluated here	Explicitly addresses tokenization

SPRC is therefore best interpreted as a target-conditioned sparse delta format coupled to a canonical integer sampler. Its current evidence supports smaller records than fixed block repair and the matched unquantized variants on one frozen bundle. The top-two comparison isolates a small contract advantage; the top-16-cap result shows that wider unquantized support sharply increases corrections in this target stack. These data do not establish global optimality, replay security or text-channel synchronization.

Supplementary Figure 1



Supplementary Figure 1 | Official SparSamp Table 2 compatibility reproduction. Published entropy-utilization values and the local compatibility reproduction across message block lengths 2-1023 at top-p=1.0. Error bars show 95% intervals from 10,000 context bootstrap resamples. The 512 and 1023 configurations contained three and four budget-exhausted trials, respectively. These trials are retained in the configured-trial count and have no capacity value.

References

[1] Yuan, J. *et al.* Understanding and mitigating numerical sources of nondeterminism in LLM inference. *arXiv* 2506.09501 (2025).

[2] Karvonen, A. *et al.* DiFR: inference verification despite nondeterminism. *arXiv* 2511.20621 (2025).

[3] Wang, Y. *et al.* SparSamp: efficient provably secure steganography based on sparse sampling. In *34th USENIX Security Symposium* 6817-6835 (USENIX Association, 2025).

[4] Yan, R. & Murawaki, Y. Efficient provably secure linguistic steganography via range coding. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics* 890-907 (Association for Computational Linguistics, 2026). `https://doi.org/10.18653/v1/2026.acl-long.39`.

[5] Cao, W., Wang, Y. & Hu, D. Breaking the "provable security": detecting finite-precision artifacts in LLM-based steganography via low-probability vanishing. In *Findings of the Association for Computational Linguistics: ACL 2026* 20262-20274 (Association for Computational Linguistics, 2026). `https://doi.org/10.18653/v1/2026.findings-acl.1013`.

[6] Qwen Team. Qwen2.5 technical report. *arXiv* 2412.15115 (2024).

[7] Pang, K. & Bai, M. Provably secure steganography based on list decoding. *arXiv* 2604.21394 (2026).

[8] Bar-Lev, D., Farnoud, F. & Gabrys, R. An additive approximation scheme for generating dyadic codings for the outputs of an LLM. *arXiv* 2605.05837 (2026).

[9] Wang, Y. *et al.* ReTokSync: self-synchronizing tokenization disambiguation for generative linguistic steganography. *arXiv* 2604.25486 (2026).